Libraries
Home: http://www.haskell.org/ghc/docs/latest/html/libraries/
Basic libraries: https://hackage.haskell.org/package/base
Prelude: https://hackage.haskell.org/package/base/docs/Prelude.html

```
class Enum a where
  succ :: a -> a
  pred :: a -> a
  toEnum :: Int -> a
  fromEnum :: a -> Int
  enumFrom :: a -> [a]
  enumFromThen :: a -> a -> [a]
  enumFromTo :: a -> a -> [a]
  enumFromThenTo :: a -> a -> a -> [a]
  {-# MINIMAL toEnum, fromEnum #-}

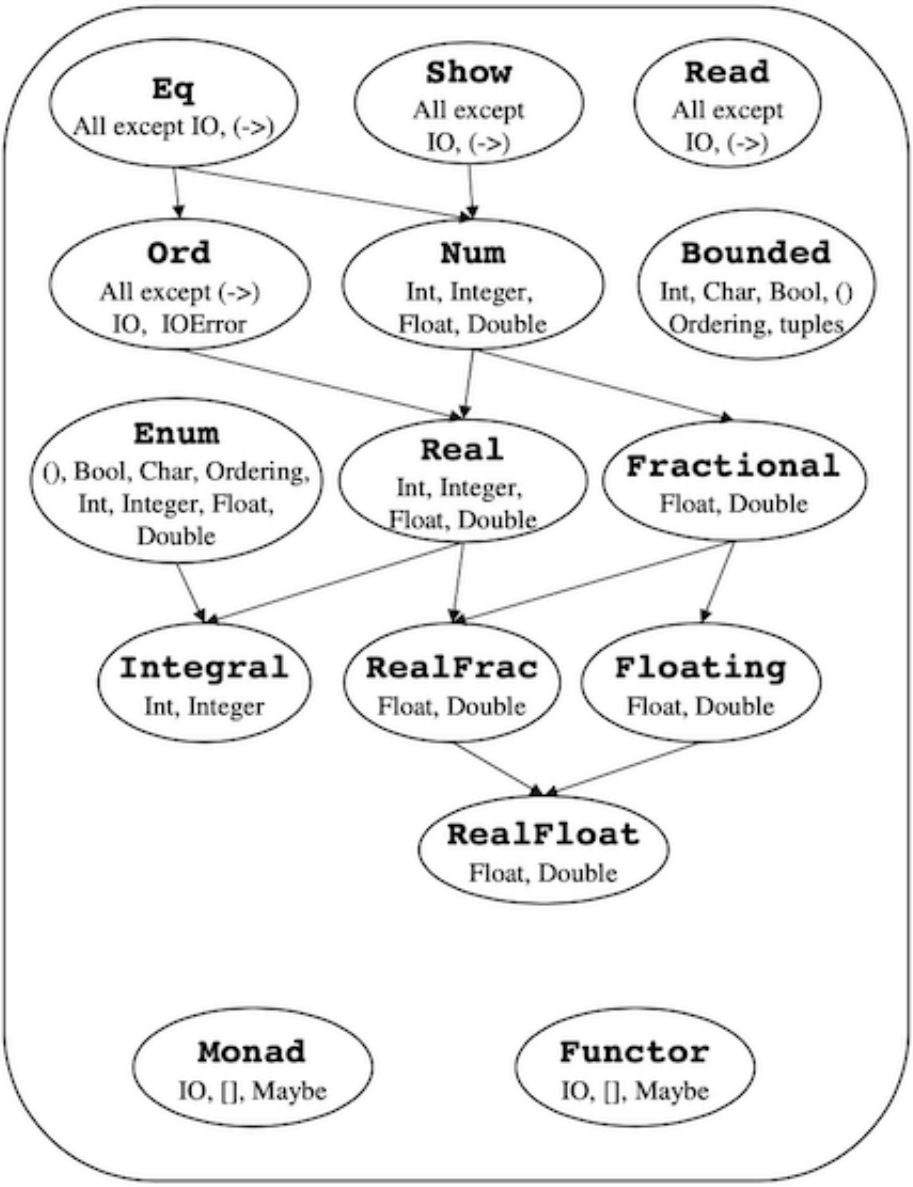
class Bounded a where
  minBound :: a
  maxBound :: a
  {-# MINIMAL minBound, maxBound #-}

error :: GHC.Stack.Types.HasCallStack => [Char] -> a
undefined :: GHC.Stack.Types.HasCallStack => a

seq :: a -> b -> b
asTypeOf :: a -> a -> a

getLine::IO String
getChar::IO Char
getContents::IO String
putChar::Char -> IO ()
putStrLn::String -> IO ()
putStr::String -> IO ()
print::Show a => a -> IO ()
```

Overview




General utils

```
---- Basic
--
error "OOPS"                      -- throw error with message
errorWithoutStackTrace "OOPS"  --
undefined
```

```
-- with stacktrace
*** Exception: OOPS
CallStack (from HasCallStack):
  error, called at <interactive>:206:1 in interactive:Ghci94

-- without stacktrace
*** Exception: OOPS

-- undefined
*** Exception: Prelude.undefined
CallStack (from HasCallStack):
  error, called at libraries/base/GHC/Err.hs:74:14 in base:GHC.Err
  undefined, called at <interactive>:208:1 in interactive:Ghci94
```

```
-- seq: sequence of expressions
-- Enforces x to be evaluated
-- Similar to const x y.
seq :: a -> b -> b
seq x y            -- ryeturns y.
y                  -- same
const y x          -- same

one = asTypeOf 1   -- Same as const, but must be same type
one 2              -- 1
one 'a'            -- ERROR Different type
n = maxBound             -- ERROR. Unknown type
n = asTypeOf 1 maxBound  -- OK


---- IO utils
--
-- e.g.
do
x <- getLine
putStrLn $ "value" ++ x

-- get: i.e. stdin
getLine::IO String
getChar::IO Char
getContents::IO String
interact:: (String -> String) -> IO ()
interact (\x -> "value=" ++ x)

-- put: i.e. stdout
putChar::Char -> IO ()
putStrLn::String -> IO ()        -- stdout on new line
putStr::String -> IO ()          -- no new line
print::Show a => a -> IO ()
putStr "Hello"
print 1
print (show 1)               -- same
```

Enum
Data.Enum: https://hackage.haskell.org/package/base/docs/Data-Enum.html

```
---- class Enum
```

Bounded

**Bounded**
Represents a type with a **max** and **min bound.**
i.e. **range** values

```
---- class Bounded
minBound :: Int        -- -9223372036854775808
```

```
maxBound :: Int        -- 9223372036854775807
minBound :: Char       -- '\Nul' i.e. 0
maxBound :: Char       -- '\1114111' Unicode?
```

==========

Boolean

Data.Bool: https://hackage.haskell.org/package/base/docs/Data-Bool.html
Data.Eq: https://hackage.haskell.org/package/base/docs/Data-Eq.html
Data.Ord: https://hackage.haskell.org/package/base/docs/Data-Ord.html

Data.Bool

```
data Bool = False | True
(&&) :: Bool -> Bool -> Bool
(||) :: Bool -> Bool -> Bool

not :: Bool -> Bool
otherwise :: Bool
bool :: a -> a -> Bool -> a
```

Data.Eq

```
class Eq a where
  (==) :: a -> a -> Bool
  (/=) :: a -> a -> Bool
```

Data.Ord

```
class Eq a => Ord a where
  compare :: a -> a -> Ordering
  (<) :: a -> a -> Bool
  (<=) :: a -> a -> Bool
  (>) :: a -> a -> Bool
  (>=) :: a -> a -> Bool
  max :: a -> a -> a
  min :: a -> a -> a
data Ordering
  = LT
  | EQ
  | GT
newtype Down a = Down { getDown :: a }
comparing :: Ord a => (b -> a) -> b -> b -> Ordering
clamp :: Ord a => (a, a) -> a -> a
```

Eq class laws

```
Eq class laws
(x == x) == True                        Reflexivity
```

```
(x == y) == (y == x)                    Symmetry
if x == y and y == z then x == z        Transitivity
if x == y then f x == f y               Extensionality (Preservation)
 (if range of f is also Eq)
x \= y == not (x == y)                  Negation
```

Ord class laws

```
x <= y || y <= x == True            Comparability
if x <= y and y <= z then x <= z    Transitivity
x <= x == True                      Reflexivity
if x <= y and y <= x then x == y    Anti symmetry
```

```
---- Boolean
(||), (&&) :: Bool -> Bool -> Bool
x || y
x && y
not x
otherwise        -- same as True


---- Comparison
--
(==),(/) :: Eq a => a -> a -> Bool
x == y
x \= y

(<), (<=), (>), (>=) :: Ord a => a -> a -> Bool
x < y
x > y
x >= y
x <= y

max,min :: Ord a => a -> a -> a
max x y
min x y


---- Compare function: for custom comparison functions
--  e.g. when sorting


compare :: Ord a => a -> a -> Ordering
compare x y
compare 1 2    -- LT  1 < 2
compare 1 1    -- EQ  1 == 1
compare 2 1    -- GT  2 > 1

Data.Ord.comparing :: Ord a => (b -> a) -> b -> b -> Ordering
 -- Same as compare, but pick value to compare
comparing id                -- same as compare
comparing fst
```

```
comparing fst (1,0) (2,0)   -- compare (fst (1,0)) (fst (2,0))=LT

-- e.g.
Data.List.sortBy (comparing fst)      -- useful in list 'By'
functions
Data.List.maximumBy (comparing fst)


----  Misc utils
-- clamp: ensure value is within bound
Data.Ord.clamp :: Ord a => (a, a) -> a -> a
clamp (0,10) x      -- 0 if x < 0, 10 if x >10, x otherwise
clamp (0,10) (-1)   -- 0
clamp (0,10) 0      -- 0
clamp (0,10) 2      -- 2
```

==========

Numerical

Prelude: https://hackage.haskell.org/package/base/docs/Prelude.html
Data.Ratio: https://hackage.haskell.org/package/base/docs/Data-Ratio.html
GHC.Num: https://hackage.haskell.org/package/base/docs/GHC-Num.html
GHC.Float: https://hackage.haskell.org/package/base/docs/GHC-Float.html
GHC.Real: https://hackage.haskell.org/package/base/docs/GHC-Real.html

Definitions

```
-- GHC.Num
class (Eq a, Show a) => Num a where
  (+),(-),(*) ::a->a->a ::a->a
  negate ::a->a
  abs, signum ::a->a
  fromInteger :: Integer -> a

-- GHC.Real
class (Num a, Ord a) => Real a where
  toRational :: a -> Rational

-- GHC.Real
class (Real a, Enum a) => Integral a where
  quot, rem, div, mod :: a -> ::a-> ::a->
  quotRem, divMod :: a -> a
  toInteger :: a -> Integer

-- GHC.Real
class (Num a) => Fractional a where
  (/) :: a -> a -> a
  recip :: a -> a
  fromRational :: Rational -> a

-- GHC.Float
class (Fractional a) => Floating a where
  pi :: a
```

```
  exp, log, sqrt :: a -> a
  (**), logBase :: a -> a -> a
  sin, cos, tan :: a -> a
  asin, acos, atan :: a -> a
  sinh, cosh, tanh :: a -> a
  asinh, acosh, atanh :: a -> a

-- GHC.Float
class (Real a, Fractional a) => RealFrac a where
  truncate, floor, ceiling, round :: Integral b => a -> b
  properFraction :: Integral b => a -> (a,b)


fromIntegral :: (Integral a, Num b) => a -> b
realToFrac :: (Real a, Fractional b) => a -> b

^ :: (Num a, Integral b) -> a -> b -> a
^^ :: (Fractional a, Integral b) -> a -> b -> a

even :: Integral a => a -> Bool
odd :: Integral a => a -> Bool
gcd :: Integral a => a -> a -> a
lcm :: Integral a => a -> a -> a

recip :: Fractional a => a -> a

floatRadix:: RealFloat a -> Integer
floatDigits:: RealFloat a -> Int
floatDigits:: RealFloat a -> (Int,Int)
decodeFloat :: RealFloat a => a -> (Integer, Int)
encodeFloat :: RealFloat a => Integer -> Int -> a
scaleFloat :: RealFloat a => Int -> a -> a
exponent :: a -> Int
sgnificant :: a -> a

isInfinite,isNan,isNegativeZero, isDenormalized,isIEEE :: RealFloat
a => a -> Bool
```

4 main numeric data types
- **Int**: bounded integer
- **Integer**: unbounded integer
- **Float**: single precision float
- **Double**: double **point** float

Data types

```
-- Numbers
-- data Int (Eq,Ord,Read,Show,Enum,Bounded,Num,Real,Integral)
-- data Integer (Eq,Ord,Read,Show,Enum,Num,Real,Integral)
-- data
Float
(Eq,Ord,Read,Show,Enum,Num,Real,Fractional,Floating,RealFrac,RealFl
```

```
oat)
-- data
Double
(Eq,Ord,Read,Show,Enum,Num,Real,Fractional,Floating,RealFrac,RealFl
oat)
1::Int            -- bounded integer
1::Integer        -- unbounded integer
1.0::Float
1.0::Double


-- Data.Ratio
-- Ref: https://hackage.haskell.org/package/base/docs/Data-
Ratio.html
-- Represents a ratio of 2 numbers
--
-- data Ratio (Eq,Ord,Read,Show,Num,Real,Fractional,RealFrac)
(1 % 2)::Ratio Int           -- 1/2
-- rational numbers
type Rational = Ratio Int    -- dfn
(1 % 2)::Rational            -- same
```

Functions

Main levels of numeric classes

- **Num**: numbers as a **ring**
  i.e. addition +, additive inverse -, multiplication *
- **Real**: rational and irrational numbers, total **ordering** <>=
- **Integral**: integers
  e.g. integer division: div, mod, quot, rem
- **Fractional**: real number division /
- **RealFrac**: convert to (approximate) integer e.g. floor, ceiling, round
- **Floating**: common real functions
  e.g. exp, log, sin, cos
- **RealFloat**: machine representation of numbers
  e.g. radix, NaN, Infinity,

```
-- Hierarchy
class Num a
class (Num a, Ord a) => Real a
class Num a => Fractional a
class (Real a, Enum a) => Integral a                    Int, Integer
class (Real a, Fractional a) => RealFrac a
class Fractional a => Floating a
class RealFrac a => RealFloat a                         Double, Float
```

```
---- Conversions
--
```

```
Main idea

All conversions pass the data through an intermediate unbounded
```

```
type
That is, either
 a) data Integer: i.e. unbounded integer if converting to an int
type, OR
 b) type Rational: i.e. unbounded float as a ratio of integers, if
converting to a float type

int -> int type          x => Integer => y
int -> float type        x => Rational => y
float -> float type      x => Rational => y
float -> int type        x => round/truncate
```

```
-- Integer type conversions
toInteger :: Integral a -> Integer      -- bounded to unbounded
integer
fromInteger :: Num a => Integer -> a    -- unbounded integer to any
number
(fromInteger 1)::Int                    -- 1::Int
(fromInteger 1)::Double                 -- 1.0::Int

fromIntegral :: (Integral a, Num b) => a -> b    -- any integer ->
any number
fromIntegral = fromInteger . toInteger           -- definition

-- Floating point type conversions
toRational :: Real a => a -> Ratio                -- any number ->
rational
fromRational:: Fractional a => Rational -> a      -- rational ->
any floating point type
toRational 1            -- 1 % 1
toRational 0.5          -- 1 % 2
toRational (sqrt 2)     -- 6369051672525773 % 4503599627370496
(fromRational (1%2))::Float         -- 0.5::Float
(fromRational (1%2))::Double        -- 0.5::Double

realToFrac :: (Real a, Fractional b) => a -> b     -- any number ->
any floating point type
realToFrac = fromRational . toRational             -- definition

-- approximations
-- Often used to convert floating point type -> integer type
floor,ceiling,truncate,round :: (RealFrac a, Integral b) => a -> b
floor x            -- round to lower
ceiling x          -- round to higher
truncate x         -- cut off fractional part
round x            -- nearest integer
```

```
           floor        ceiling        truncate        round
-2.9        -3           -2              -2             -3
-2.1        -3           -2              -2             -2
2.1          2            3               2              2
2.9          2            3               2              3
```

```
---- Arithmetic operations
--
-- ring operations
(+),(-),(*) :: Num a => a -> a -> a
x + y
x - y
x * y
-x                  -- negative number
subtract 1 2        -- same as (-). Mainly for resolving ambiguity
due to - as both negative number and subtraction

-- note: - as negative number vs - as subtraction
(+1)            -- function x+1
(-1)            -- NOT x-1 but negative number -1
(+1) 2          -- 2+1=3
(-1) 2          -- ERROR
(subtract 1) 2  -- OK 2-1=1

-- sign
negate,abs,num :: Num a => a -> a
negate x            -- same as (-x)
abs x
signum x            -- -1,0,1

-- division
(/) :: Fractional a => a -> a -> a
5 / 2            -- 2.5
5 / 0            -- Infinity


-- exponents
** :: Floating => a -> a -> a
2 ** 2            -- 4
2.1 ** 2          -- 4.41    factional base
2 ** 2.1          -- 4.287...  fractional exponent
2 ** (-2.1)       -- 0.2332... negative exponent

-- integer exponents
^ :: (Num a, Integral b) -> a -> b -> a              -- non
negative integer exponents only
2 ^ 2               -- 4
2.1 ^ 2             -- 4.41
2 ^ 0               -- 0
2 ^ (-1)            -- ERROR
2 ^ (2.1)           -- ERROR

^^ :: (Fractional a, Integral b) -> a -> b -> a      -- any integer
exponent
2 ^^ 2               -- 4
2.1 ^^ 2             -- 4.41
2 ^^ 0               -- 0
```

```
2 ^^ (-1)            -- 0.5
2 ^^ (2.1)           -- ERROR


-- modular arithmetic
div,mod,quot,rem :: Integral a => a -> a -> a
div x y              -- x (mod y)
mod x y
quot x y             -- quotient, remainder of x/y
rem x y
quotRem,divMod :: Integral a => a -> a -> (a,a)
```

```
div,mod: modular arithmetic
   x = dy + r  such that  x = r (mod y)                d=(div x
y), b=(mod x y)
   i.e. (mod x y) has the same sign as y

quot,rem: remainder
   x = dy + r  such that  r = remainder of x/y         d=(quot x
y), b=(rem x y)
   i.e. (rem x y) has the same sign as x



                  x = (div x y) y + (mod x y)           x = (quot
x y) y + (rem x y)
                i.e. signum x = signum (mod x y)        i.e. signum
x = signum (rem x y)
x= 5  y= 3                 x =   y+2                            x
=  y+2
x= 5  y=-3                 x = -2y-1                            x
= -y+2
x=-5  y= 3                 x = -2y+1                            x
= -y-2
x=-5  y=-3                 x =   y-2                            x
=  y-2
```

```
---- Utilities
--
-- integer
even :: Integral a => a -> Bool
odd :: Integral a => a -> Bool
gcd :: Integral a => a -> a -> a
lcm :: Integral a => a -> a -> a
properFraction :: (Integral b, RealFrac a) => b -> (b,a)

-- real numbers
recip :: Fractional a => a -> a   -- reciprocal 1/x
recip 4                           -- 0.25
recip 0                           -- Infinity
pi :: Floating a => a
exp :: Floating a => a -> a
exp x
```

```
sqrt x
log x
logBase x y

-- trigonometric
sin, cos, tan :: Floating a => a -> a
sin x
cos x
tan x
...etc
```

Machine representation

Defined by **RealFloat** type class

```
class  (RealFrac a, Floating a) => RealFloat a  where
    floatRadix          :: a -> Integer
    floatDigits         :: a -> Int
    floatRange          :: a -> (Int,Int)
    decodeFloat         :: a -> (Integer,Int)
    encodeFloat         :: Integer -> Int -> a
    exponent            :: a -> Int
    significand         :: a -> a
    scaleFloat          :: Int -> a -> a
    isNaN               :: a -> Bool
    isInfinite          :: a -> Bool
    isDenormalized      :: a -> Bool
    isNegativeZero      :: a -> Bool
    isIEEE              :: a -> Bool
    atan2               :: a -> a -> a
    ...
```

```
---- Machine representation of real numbers
-- i.e. floating point representation
--
--
-- A number n is represented as
--   n = s * r^e
--             s = sigificand
--             r = radix i.e. base. Usually 2
--             e = exponent  i.e. fractional part
--
-- Data type info: all constant
floatRadix:: RealFloat a -> Integer
floatRadix (1::Float)        -- 2.
floatRadix (100::Float)      -- 2.
floatRadix (1::Double)       -- 2
floatDigits:: RealFloat a -> Int
floatDigits (1::Float)       -- e.g. 24.  precision, i.e.
significant digits, of the data
                             -- 24 significant binary digits = 6~9
```

```
significant decimal digits
floatDigits (1::Double)      -- e.g. 53 significant binary digits
floatDigits:: RealFloat a -> (Int,Int)
floatRange (1::Float)        -- (-125,128). Min/max exponents
                             -- i.e. smallest number~2^-125
 largest number~2^128
floatRange (1::Double)       -- (-1021,1024)

-- Floating point representation
decodeFloat :: RealFloat a => a -> (Integer, Int)
encodeFloat :: RealFloat a => Integer -> Int -> a
scaleFloat :: RealFloat a => Int -> a -> a
decodeFloat (0.1::Float)                          -- (13421773,-27)
i.e. machine representation 0.1 = 13421773 * 2^(-27)
(encodeFloat 13421773 (-27))::Float               -- 0.1 i.e.
inverse
(scaleFloat (-27) 13421773)::Float                -- 0.1 i.e.
encodeFloat flipped
exponent (0.1::Float)                             -- -3
significand  (0.1::Float)                         -- 0.8

decodeFloat (0.1::Double)                         --
 (7205759403792794,-56) i.e. representation 0.1 =
7205759403792794 * 2^(-56)
(encodeFloat 7205759403792794 (-56))::Double      -- 0.1
(scaleFloat (-56) 7205759403792794)::Double       -- 0.1
exponent (0.1::Double)                            -- -3
significand  (0.1::Double)                        -- 0.8

-- representation utils
isInfinite,isNan,isNegativeZero, isDenormalized,isIEEE :: RealFloat
a => a -> Bool
isInfinite (1/0)        -- True
isNan x
isNegativeZero (-0)     -- True
isDenormalized x
isIEEE x
...
```

==========

Maybe, Either

Prelude: https://hackage.haskell.org/package/base/docs/Prelude.html
Data.Maybe: https://hackage.haskell.org/package/base/docs/Data-Maybe.html
Data.Either: https://hackage.haskell.org/package/base/docs/Data-Either.html

Data.Maybe

```
data Maybe a = Nothing | Just a

isJust :: Maybe a -> Bool
isNothing :: Maybe a -> Bool
fromMaybe :: a -> Maybe a -> a
fromJust :: HasCallStack => Maybe a -> a
```

```
maybe :: b -> (a -> b) -> Maybe a -> b
catMaybes :: [Maybe a] -> [a]
maybeToList :: Maybe a -> [a]
listToMaybe :: [a] -> Maybe a
mapMaybe :: (a -> Maybe b) -> [a] -> [b]
```

Data.Either

```
data Either a b = Left a | Right b

isLeft :: Either a b -> Bool
isRight :: Either a b -> Bool
fromLeft :: a -> Either a b -> a
fromRight :: b -> Either a b -> b
either :: (a -> c) -> (b -> c) -> Either a b -> c
lefts :: [Either a b] -> [a]
rights :: [Either a b] -> [b]
partitionEithers :: [Either a b] -> ([a], [b])
```

**Maybe a**
**Either a b**
Both Maybe and Either represent **success/failure**

- **Maybe**: analogous to **early exit.**
e.g. early exit with validation

```
// In JavaScript
function doSomething(s1, s2) {
  const x = parseInt(s1);
  if (isNan(x)) return null;
  const y = parseInt(s2);
  if (isNan(y)) return null;

  return x * y;
}



--  With Maybe
import Text.Read (readMaybe)
doSomething :: String -> String -> Maybe Int
doSomething s1 s2 = do
  x <- readMaybe s1
  y <- readMaybe s2
  return x * y

doSomething "1" "2"     -- Just 2
doSomething "1" "a"     -- Nothing
doSomething "a" "2"     -- Nothing
```

- **Either**: analogous to throwing an **error**.

e.g. error with validation

```
// In JavaScript
function doSomething(s1, s2) {
  const x = parseInt(s1);
  if (isNan(x)) throw new Error(`Failed to parse ${s1}`);
  const y = parseInt(s2);
  if (isNan(y)) throw new Error(`Failed to parse ${s12}`);
  return x * y;
}


-- In Haskell with Either
import Text.Read (readEither)
doSomething :: String -> String -> Either String Int
doSomething s1 s2 = do
 x <- readEither s1
 y <- readEither s2
 return x * y

doSomething "1" "2"     -- Right 2
doSomething "1" "a"     -- Left "Prelude.read: no parse"
doSomething "a" "2"     -- Left "Prelude.read: no parse"
```

e.g. chaining computations with errors

```
// In JavaScript
function doSomething (x) {
 y1 = compute1(x);
 y2 = compute2(y1);               // called only if no error throw
by compute1
 y3 = compute3(y2);               // called only if no error throw
by compute1,2
 return y1 + y2 + y3;             // called only if no error throw
by all compute1,2,3
}


-- In Haskell
doSomething :: Int -> Either String Int
doSomething = do
 y1 <- compute1 x
 y2 <- compute2 y1                -- called only if compute1
succeeded
 y3 <- compute3 y2                -- called only if compute1,2
succeeded
 return $ y1 + y2 + y3            -- called only if compute1,2,3
succeeded
```

Data type

```
---- data Maybe a (Eq, Ord, Read1, Show1, Enum, Bounded, Functor,
Applicative, Monad)
--
Just 1        -- Num a => Just a
Nothing

-- use Data.Maybe.maybe instead of case for handling
maybe :: b -> (a->b) -> Maybe a -> b
case r of                          -- instead of
  Just x = f x
  Nothing = x0
maybe x0 f r                       -- same without case


---- data Either a b (Eq, Ord, Read2, Show2, Enum, Bounded,
Functor, Applicative, Monad)
--
Left 1        -- Num a => Either a b
Right 1       -- Num b => Either a b

-- use Data.Either.either instead of case for handling
either :: (a -> c) -> (b -> c) -> Either a b -> c
case r of                          -- instead of
  Left v -> f v
  Right v -> g v
either f g r                       -- same



---- Data.Maybe utils
--
import Data.Maybe
-- Checks
isJust $ Just 1          -- True
isJust Nothing           -- False
isNothing $ Just 1       -- False

-- value
fromJust :: Maybe a -> a              -- value or error
fromMaybe :: a -> Maybe a -> a        -- value or default value
maybeToList :: Maybe a -> [a]         -- value or empty list
maybe :: b -> (a->b) -> Maybe a -> b  -- fromMaybe with map
fromJust $ Just 5        -- 5
fromJust Nothing         -- Error
fromMaybe 0 $ Just 5     -- 5
fromMaybe 0 $ Nothing    -- 0
maybeToList $ Just 5     -- [5]
maybeToList Nothing      -- []
maybe 0 (*2) $ Just 5    -- 10
maybe 0 (*2) Nothing     -- 0

-- e.g. maybe = fromMaybe + map
maybe x0 f m
```

```
fromMaybe x0 (f<$>m)     -- same



-- list
catMaybes :: [Maybe a] -> [a]
listToMaybe :: [a] -> Maybe a
mapMaybe ::(a -> Maybe a) -> [a] -> [a]
catMaybes []                            -- []
catMaybes [Just 1, Nothing, Just 2]     -- [1,2]
listToMaybe [1,2]                       -- Just 1. Just <first
item>
listToMaybe []                          -- Nothing
justEven x = if even x then (Just $ 2*x) else Nothing
mapMaybe justEven [1,2]                 -- [2]. Apply map, filter
out Nothing
mapMaybe f l
[x|x <- l, isJust (isEven x)]           -- same



---- Data.Either utils
--
import Data.Either

-- checks
isLeft $ Left 1     -- True
isLeft $ Right 1    -- False
isRight             -- opposite

-- value
fromLeft :: a -> Either a b -> a
fromRight :: b -> Either a b -> b    -- opposite
fromLeft 0                           -- Value of Left, default=0
otherwise
fromLeft 0 $ Left 1                  -- 1
fromLeft 0 $ Right 1                 -- 0
fromLeft 0 $ Left 'a'                -- ERROR. Different type
fromRight 0

either :: (a -> c) -> (b -> c) -> Either a b -> c
either f g x
case x of                    -- same
  Left v -> f v
  Right v -> g v
either (*2) (*3)             -- apply (*2) if Left, (*3) if Right
either (*2) (*3) $ Left 10   -- 20
either (*2) (*3) $ Right 10  -- 30

-- list
lefts [Left 1, Right 10, Left 2]    -- [1,2]
rights [Left 1, Right 10, Left 2]   -- [10]
partitionEithers [Left 1, Right 10, Left 2]     -- ([1,2],[10])
```